

EDA — Dataset de mandioca · Clasificación de enfermedades (5 clases)

Trabajo Final Integrador — IC522 · Inteligencia Computacional · WLODECK, Fabricio Joaquín

Objetivo: analizar la integridad, composición y calidad del dataset de imágenes de hojas de mandioca para preparar el pipeline de clasificación de la consigna.

Diccionario de clases (usado en todas las salidas):

Código	Nombre legible	Descripción
0	CBB	Cassava Brown Streak Disease (bacteriosis / CFMV)
1	CBSD	Cassava Brown Streak Disease (rayas marrones)
2	CGM	Cassava Green Mosaic (moteado verde)
3	CMD	Cassava Mosaic Disease (mosaico)
4	Saludable	Hoja sin síntomas

Cómo ejecutar

- **VS Code:** abrir la carpeta del proyecto, instalar Python y Jupyter, confiar en la carpeta (Workspace Trust), abrir el `.ipynb` y *Run All* (o ejecutar con nbconvert, ver al final).
- **Google Colab:** *Archivo* → *Subir a Google Drive* o subir el notebook; subir `labels.csv` (y opcionalmente `mandioca.zip` para las celdas de imágenes, que se desactivan solas si no encuentran la carpeta).

Entregables: `01_EDA_mandioca.ipynb` (ejecutado) + `.html` + `.pdf`.

Las celdas de **imágenes** (Fase 5) se autodesactivan si no existe la carpeta de imágenes.

```
In [1]: import os, re, time, random, hashlib
from pathlib import Path
from collections import Counter, defaultdict

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
```

```

NOMBRE = {
    0: "CBB (bacteriosis)",
    1: "CBSD (rayas marrones)",
    2: "CGM (moteado verde)",
    3: "CMD (mosaico)",
    4: "Saludable",
}
PALETA = {0: "#D73027", 1: "#FC8D59", 2: "#FCC163", 3: "#4575B4", 4: "#1A9850"}
ORDEN = sorted(NOMBRE)

pd.set_option("display.max_columns", 20)
pd.set_option("display.width", 140)
pd.set_option("display.max_colwidth", 60)
plt.rcParams.update({
    "figure.dpi": 110,
    "savefig.dpi": 110,
    "figure.facecolor": "white",
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 10,
})

def pct(s):
    return (100 * s / s.sum()).round(2)

print("Semilla:", SEED, "| pandas", pd.__version__, "| numpy", np.__version__)

```

Semilla: 42 | pandas 3.0.6 | numpy 2.5.3

Fase 0 — Entorno y carga de datos

Detección de entorno (Colab / local) y ubicación de `labels.csv` con rutas candidatas + respaldo absoluto.

```

In [2]: def detectar_entorno():
    try:
        import google.colab # noqa: F401
        return "colab"
    except Exception:
        return "local"

ENTORNO = detectar_entorno()
print("Entorno detectado:", ENTORNO)

CANDIDATOS = [
    Path("datos/labels.csv"),
    Path("labels.csv"),
    Path("../datos/labels.csv"),
    Path(r"C:\Users\Fabricio\Desktop\final_inteligencia\datos\labels.csv"),
]

if ENTORNO == "colab":
    print("En Colab subi 'labels.csv' por el panel de archivos (o ejecuta files.upl
    try:

```

```

from google.colab import files
if not any(p.exists() for p in CANDIDATOS):
    uploaded = files.upload()
    CANDIDATOS.insert(0, Path(list(uploaded)[0]))
except Exception as e:
    print("No se pudo abrir files.upload():", e)

RUTA_CSV = next((p for p in CANDIDATOS if p.exists()), None)
assert RUTA_CSV is not None, "No se encontro labels.csv en ninguna ruta candidata"
df = pd.read_csv(RUTA_CSV)
print("CSV:", RUTA_CSV.resolve())
print("Shape:", df.shape)

```

Entorno detectado: local

CSV: C:\Users\Fabrizio\Desktop\final_inteligencia\datos\labels.csv

Shape: (26337, 3)

Fase 1 — Identidad del dataset y diccionario

Primer contacto: dimensiones, tipos, nulos y muestra de filas.

```

In [3]: print("Forma:", df.shape)
print("\nColumnas:", list(df.columns))
display(df.head())
display(df.tail())

tabla_diccionario = pd.DataFrame({
    "columna": ["image_id", "label", "source"],
    "tipo": [str(df.image_id.dtype), str(df.label.dtype), str(df.source.dtype)],
    "descripcion": [
        "nombre del archivo .jpg en la carpeta images/",
        "clase entera 0..4 (mapear con NOMBRE de la celda 1)",
        "fuente/campana de captura (2019 o 2020)",
    ],
    "n_unicos": [df.image_id.nunique(), df.label.nunique(), df.source.nunique()],
})
display(tabla_diccionario)

```

Forma: (26337, 3)

Columnas: ['image_id', 'label', 'source']

	image_id	label	source
0	1000015157.jpg	0	2020
1	1000201771.jpg	3	2020
2	100042118.jpg	1	2020
3	1000723321.jpg	1	2020
4	1000812911.jpg	3	2020

	image_id	label	source
26332	train-cbb-114.jpg	0	2019
26333	train-cmd-661.jpg	3	2019
26334	train-cmd-1716.jpg	3	2019
26335	train-healthy-236.jpg	4	2019
26336	train-cgm-498.jpg	2	2019

	columna	tipo	descripcion	n_unicos
0	image_id	str	nombre del archivo .jpg en la carpeta images/	26337
1	label	int64	clase entera 0..4 (mapear con NOMBRE de la celda 1)	5
2	source	int64	fuentes/campana de captura (2019 o 2020)	2

Fase 2 — Integridad y calidad (antes de mirar distribuciones)

Chequeos de nulos, duplicados de id, cruce CSV ↔ disco, duplicados exactos (SHA-1 completo), legibilidad de las imágenes y coherencia **nombre del archivo** ↔ **etiqueta**.

```
In [4]: print("Nulos por columna:\n", df.isna().sum().to_string())
print("\nimagen_id duplicados:", int(df.imagen_id.duplicated().sum()))
print("Valores de label:", sorted(df.label.unique()))
print("Valores de source:", sorted(df.source.unique()))
print("rango de label valido [0,4]:", bool(df.label.between(0, 4).all()))
print("extensiones presentes:", Counter(Path(x).suffix for x in df.imagen_id))
```

Nulos por columna:

```
imagen_id    0
label        0
source       0
```

imagen_id duplicados: 0

Valores de label: [np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4)]

Valores de source: [np.int64(2019), np.int64(2020)]

rango de label valido [0,4]: True

extensiones presentes:

```
Counter({' .jpg': 26337})
```

```
In [5]: def buscar_carpeta_imagenes():
cands = [
    Path("datos/images"),
    Path("images"),
    Path("datos/mandioca/images"),
    Path("mandioca/images"),
    Path(r"C:\Users\Fabricio\Desktop\final_inteligencia\datos\images"),
]
return next((p for p in cands if p.exists()), None)
```

```

RAIZ_IMG = buscar_carpeta_imagenes()
if RAIZ_IMG is None:
    print("AVISO: no se encontro la carpeta de imagenes.")
    print("Las fases de imagenes se omitiran (en Colab: subir mandioca.zip y descom
else:
    disco = set(os.listdir(RAIZ_IMG))
    en_csv = set(df.image_id)
    print("Carpeta:", RAIZ_IMG.resolve())
    print("Archivos en disco:", len(disco), "| image_id en CSV:", len(en_csv))
    print("Faltan en disco:", len(en_csv - disco), "| Sobran en disco:", len(disco
    print("Extensiones no .jpg:", {e for e in disco if not e.lower().endswith(".jpg
    print("Total GB:", round(sum(os.path.getsize(RAIZ_IMG / f) for f in disco) / 10

```

Carpeta: C:\Users\Fabricio\Desktop\final_inteligencia\datos\images
 Archivos en disco: 26337 | image_id en CSV: 26337
 Faltan en disco: 0 | Sobran en disco: 0
 Extensiones no .jpg: ninguna
 Total GB: 3.09

2.1 Duplicados exactos (SHA-1 completo)

Lectura de **todos** los archivos en chunks de 1 MB (≈ 30 s en este equipo para 3.09 GiB). Se reportan: archivos, hashes únicos, grupos duplicados, archivos involucrados y excedentes.

```

In [6]: if RAIZ_IMG is None:
        print("Omitido: no hay carpeta de imagenes.")
    else:
        t0 = time.time()
        hashes = defaultdict(list)
        files_all = sorted(f for f in os.listdir(RAIZ_IMG))
        for i, f in enumerate(files_all, 1):
            h = hashlib.sha1()
            with open(RAIZ_IMG / f, "rb") as fh:
                for chunk in iter(lambda: fh.read(1 << 20), b''):
                    h.update(chunk)
            hashes[h.hexdigest()].append(f)
            if i % 5000 == 0:
                print(f" {i}/{len(files_all)} archivos hasheados...")

        dups = {k: v for k, v in hashes.items() if len(v) > 1}
        print(f"Archivos procesados: {len(files_all)}")
        print(f"Hashes unicos:      {len(hashes)}")
        print(f"Grupos duplicados:   {len(dups)}")
        print(f"Archivos en duplicados: {sum(len(v) for v in dups.values())}")
        print(f"Excedentes (a borrar si hubiera): {sum(len(v) - 1 for v in dups.values())}")
        for k, v in list(dups.items())[:10]:
            print("  dup:", v)
        print(f"Tiempo: {time.time() - t0:.1f} s")

```

5000/26337 archivos hasheados...
 10000/26337 archivos hasheados...
 15000/26337 archivos hasheados...
 20000/26337 archivos hasheados...
 25000/26337 archivos hasheados...

```
Archivos procesados: 26337
Hashes unicos:      26337
Grupos duplicados:  0
Archivos en duplicados: 0
Excedentes (a borrar si hubiera): 0
Tiempo: 25.8 s
```

2.2 Legibilidad de las imágenes (muestra aleatoria, semilla fija)

```
In [7]: try:
        from PIL import Image
        PIL_OK = True
    except Exception as e:
        PIL_OK = False
        print("Pillow no disponible:", e)

    if RAIZ_IMG is None or not PIL_OK:
        print("Omitido: sin carpeta de imagenes o sin Pillow.")
    else:
        t0 = time.time()
        files_all = [f for f in os.listdir(RAIZ_IMG) if f.lower().endswith(".jpg")]
        muestra = random.sample(files_all, min(3000, len(files_all)))
        modos, rotas = Counter(), []
        for f in muestra:
            try:
                with Image.open(RAIZ_IMG / f) as im:
                    modo = im.mode
                with Image.open(RAIZ_IMG / f) as im:
                    im.verify()
                modos[modo] += 1
            except Exception:
                rotas.append(f)
        print("Muestra verificada:", len(muestra))
        print("Ilegibles/corruptas:", len(rotas), rotas[:10])
        print("Modos de color:", dict(modos))
        print(f"Tiempo: {time.time() - t0:.1f} s")
```

```
Muestra verificada: 3000
Ilegibles/corruptas: 0 []
Modos de color: {'RGB': 3000}
Tiempo: 1.7 s
```

2.3 Coherencia nombre del archivo ↔ etiqueta (chequeo de filtrado de la etiqueta)

El subconjunto `source = 2019` tiene nombres `train-<tipo>-NNN.jpg`, que **contienen la clase**. Si el crosstab es diagonal, el nombre filtra la etiqueta: jamás debe usarse como feature.

```
In [8]: tipo = df.image_id.str.extract(r"^train-([a-z]+)-\d+\.jpg$")[0]
        con_patron = tipo.notna()
        print("Archivos con nombre descriptivo:", int(con_patron.sum()), "| numericos:", in
```

```

ct = pd.crosstab(tipo.fillna("(numerico 2020)"), df.label, margins=True)
ct.rename(columns=NOMBRE, inplace=True)
display(ct)

MAPEO_NOMBRE = {"cbb": 0, "cbsd": 1, "cgm": 2, "cmd": 3, "healthy": 4}
sub = df[con_patron].copy()
sub["label_esperado"] = sub.image_id.str.extract(r"^train-([a-z]+)-\d+\.\jpg$")[0].m
coherencia = float((sub.label == sub.label_esperado).mean())
print(f"Coherencia nombre->label en los descriptivos: {100 * coherencia:.2f} %")
print("Todos los descriptivos pertenecen a source 2019:",
      bool((df.loc[con_patron, "source"] == 2019).all()))

```

Archivos con nombre descriptivo: 4940 | numericos: 21397

label	CBB (bacteriosis)	CBSD (rayas marrones)	CGM (moteado verde)	CMD (mosaico)	Saludable	All
0						
(numerico 2020)	1087	2189	2386	13158	2577	21397
cbb	405	0	0	0	0	405
cbsd	0	1287	0	0	0	1287
cgm	0	0	631	0	0	631
cmd	0	0	0	2304	0	2304
healthy	0	0	0	0	313	313
All	1492	3476	3017	15462	2890	26337

Coherencia nombre->label en los descriptivos: 100.00 %
 Todos los descriptivos pertenecen a source 2019: True

Fase 3 — Composición y desbalance de clases

Normalización: todos los % de esta sección son **sobre el total de 26,337 registros**, salvo que se indique otra base (se aclara celda por celda).

```

In [9]: t_clase = df.label.value_counts().sort_index().rename_axis("label").to_frame("n")
t_clase["%"] = pct(t_clase.n)
t_clase.index = [NOMBRE[i] for i in t_clase.index]
display(t_clase)

ratio = t_clase.n.max() / t_clase.n.min()
print(f"Clase mayoritaria: {t_clase.n.idxmax()} | minoritaria: {t_clase.n.idxmin()}")
print(f"Ratio desbalance max/min: {ratio:.2f} : 1")

fig, ax = plt.subplots(figsize=(8, 4.2))
etiquetas = [NOMBRE[i] for i in ORDEN]
valores = [int(t_clase.loc[e, "n"]) for e in etiquetas]
ax.bar(etiquetas, valores, color=[PALETA[i] for i in ORDEN], edgecolor="black", lin
for x, v in zip(range(len(valores)), valores):

```

```

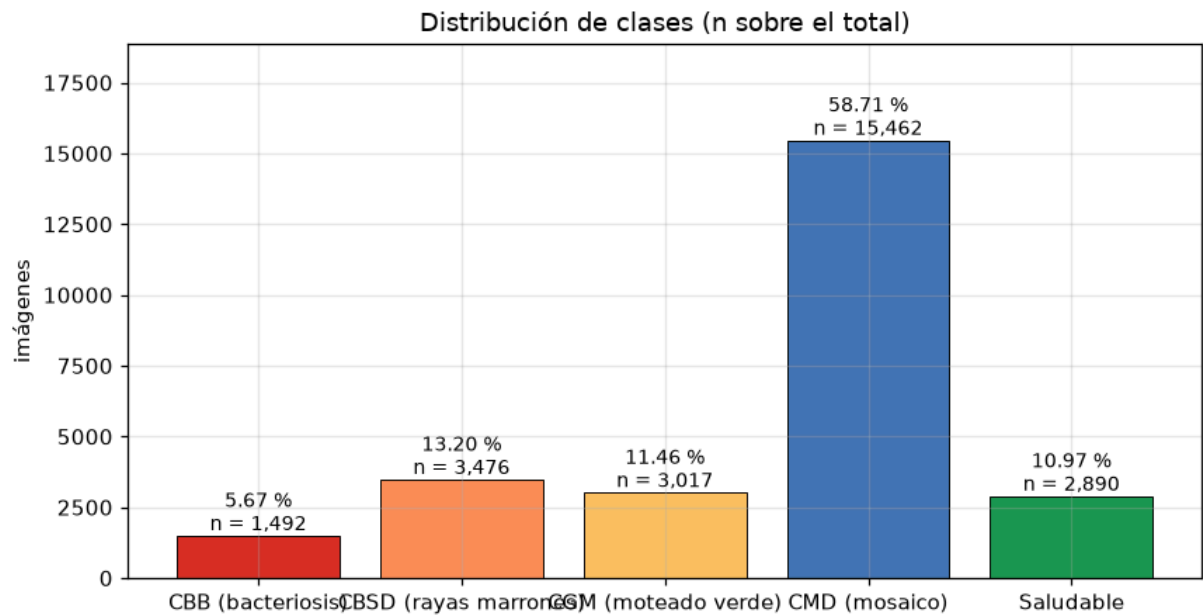
    ax.text(x, v + 200, f"{100 * v / sum(valores):.2f} %\nn = {v:,.}", ha="center",
ax.set_title("Distribución de clases (n sobre el total)")
ax.set_ylabel("imágenes")
ax.set_ylim(0, max(valores) * 1.22)
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(7.5, 7.5))
etiquetas_pie = [f"{e}\nn = {v:,.}" for e, v in zip(etiquetas, valores)]
wedges, textos, autopcts = ax.pie(
    valores,
    labels=etiquetas_pie,
    colors=[PALETA[i] for i in ORDEN],
    autopct=lambda p: f"{p:.2f} %",
    startangle=90,
    counterclock=False,
    wedgeprops={"edgecolor": "black", "linewidth": 0.8},
    pctdistance=0.72,
    labeldistance=1.08,
    textprops={"fontsize": 9},
)
for t in autopcts:
    t.set_color("black")
    t.set_fontsize(9)
    t.set_fontweight("bold")
ax.set_title("Distribución de clases – gráfico de torta\n(% sobre el total de 26,33
plt.tight_layout()
plt.show()

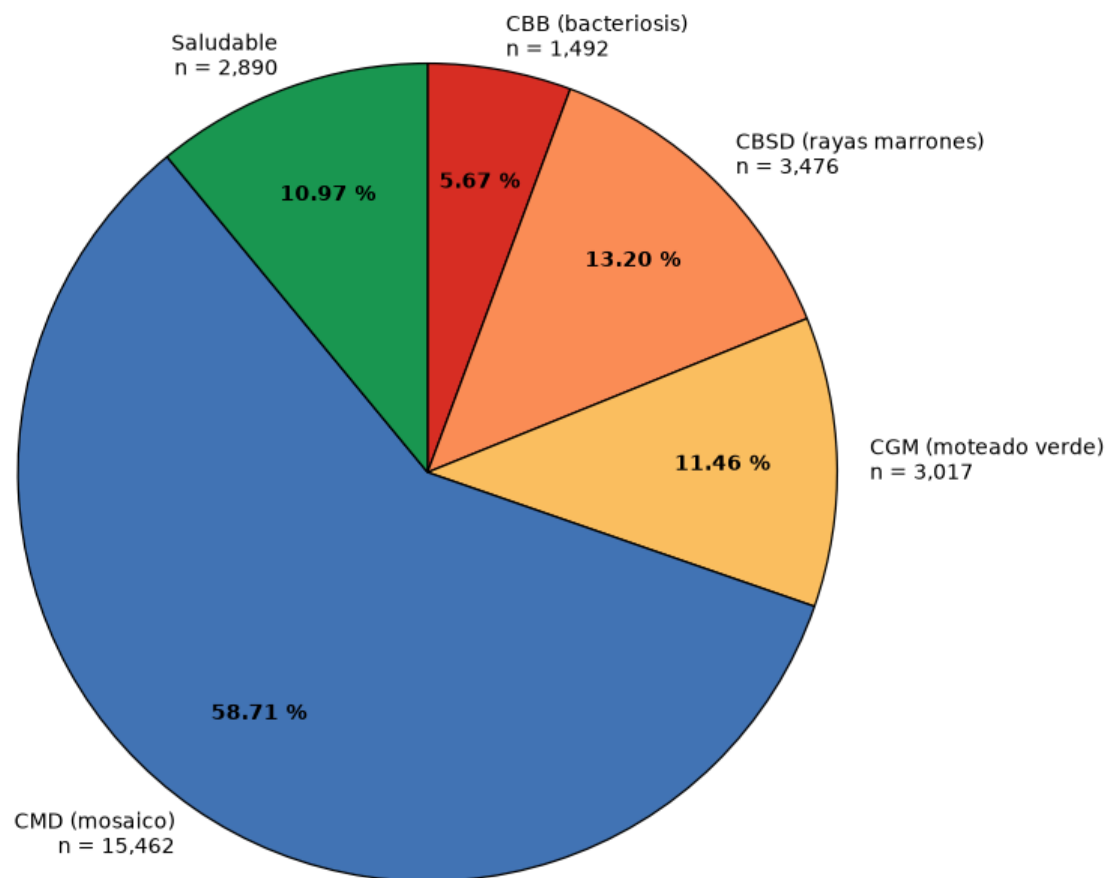
```

	n	%
CBB (bacteriosis)	1492	5.67
CBSD (rayas marrones)	3476	13.20
CGM (moteado verde)	3017	11.46
CMD (mosaico)	15462	58.71
Saludable	2890	10.97

Clase mayoritaria: CMD (mosaico) | minoritaria: CBB (bacteriosis)
Ratio desbalance max/min: 10.36 : 1



Distribución de clases — gráfico de torta
(% sobre el total de 26,337 registros)



Implicancias según la consigna

- El dataset está **muy desbalanceado** (CMD $\approx$ 59 % vs CBB $\approx$ 6 %): el *accuracy* engaña.

- Métricas a reportar en el trabajo: **macro-F1, balanced accuracy, ROC-AUC / PR-AUC y matriz de confusión normalizada por fila.**
- Costo asimétrico: lo peor es confundir **saludable** ↔ **enferma** o perder una enfermedad (falsos negativos), porque implica tratar o no tratar cultivos de forma equivocada.

```
In [10]: ct_ns = pd.crosstab(df.label, df.source, margins=True, margins_name="Total")
ct_ns.rename(index=NOMBRE, inplace=True)
print("Cruce label x source - n (absolutos):")
display(ct_ns)

ct_col = (pd.crosstab(df.label, df.source, normalize="columns") * 100).round(2)
ct_col.rename(index=NOMBRE, inplace=True)
print("Mismo cruce - % normalizado POR COLUMNA (distribucion de clases dentro de ca")
display(ct_col)

ct_row = (pd.crosstab(df.label, df.source, normalize="index") * 100).round(2)
ct_row.rename(index=NOMBRE, inplace=True)
print("Mismo cruce - % normalizado POR FILA (como se reparte cada clase entre las f")
display(ct_row)
```

Cruce label x source - n (absolutos):

	source	2019	2020	Total
label				
CBB (bacteriosis)		405	1087	1492
CBSD (rayas marrones)		1287	2189	3476
CGM (moteado verde)		631	2386	3017
CMD (mosaico)		2304	13158	15462
Saludable		313	2577	2890
Total		4940	21397	26337

Mismo cruce - % normalizado POR COLUMNA (distribucion de clases dentro de cada fuente):

	source	2019	2020
label			
CBB (bacteriosis)		8.20	5.08
CBSD (rayas marrones)		26.05	10.23
CGM (moteado verde)		12.77	11.15
CMD (mosaico)		46.64	61.49
Saludable		6.34	12.04

Mismo cruce - % normalizado POR FILA (como se reparte cada clase entre las fuentes):

	source	2019	2020
label			
	CBB (bacteriosis)	27.14	72.86
	CBSD (rayas marrones)	37.03	62.97
	CGM (moteado verde)	20.91	79.09
	CMD (mosaico)	14.90	85.10
	Saludable	10.83	89.17

Fase 4 — Fuentes de captura y sesgo de dominio

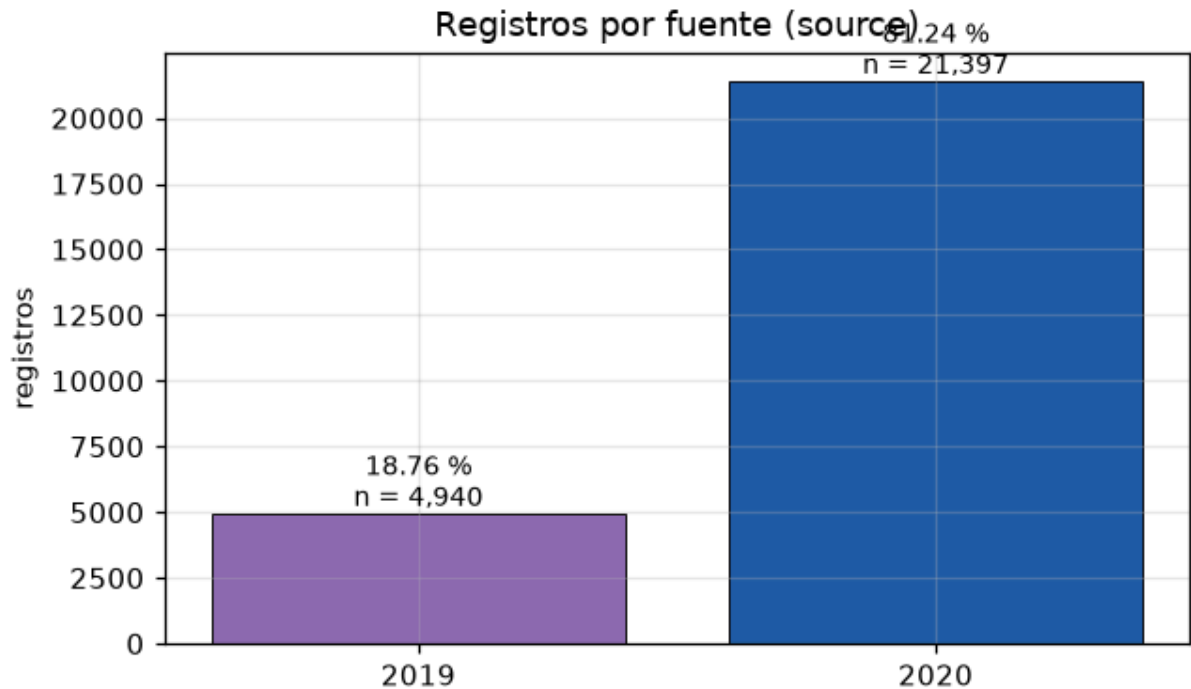
`source` separa dos campañas/años. Las de **2019** traen nombres descriptivos y las de **2020** numéricos, lo que sugiere que son datasets originales distintos (cámara, fondo e iluminación diferentes). Si el split no controla esta variable, los resultados pueden inflarse.

```
In [11]: t_fuente = df.source.value_counts().sort_index().rename_axis("source").to_frame("n")
t_fuente["%"] = pct(t_fuente.n)
display(t_fuente)

fig, ax = plt.subplots(figsize=(6, 3.6))
ax.bar([str(s) for s in t_fuente.index], t_fuente.n, color=["#8C6BB1", "#225EA8"],
       edgecolor="black", linewidth=0.6)
for i, (s, row) in enumerate(t_fuente.iterrows()):
    ax.text(i, row.n + 300, f"{row['%']:.2f} %\nn = {int(row.n):,}", ha="center", f
ax.set_title("Registros por fuente (source)")
ax.set_ylabel("registros")
plt.tight_layout()
plt.show()

presentes = pd.crosstab(df.label, df.source) > 0
print("¿Toda clase esta presente en ambas fuentes?:", bool(presentes.all().all()))
print("Mínimo de registros por (clase, fuente):", int(pd.crosstab(df.label, df.sour
```

	n	%
source		
2019	4940	18.76
2020	21397	81.24



¿Toda clase esta presente en ambas fuentes?:

True

Mínimo de registros por (clase, fuente): 313

Fase 5 — Análisis de las imágenes (núcleo del EDA)

Esta fase **se autodessactiva** si no hay carpeta de imágenes (en Colab: subir `mandioca.zip` y descomprimir). Analiza dimensiones, aspecto, brillo/contraste por clase, montajes visuales por clase e imágenes no informativas.

```
In [12]: disp_img = RAIZ_IMG is not None and PIL_OK
if not disp_img:
    print("OMITIDO: sin carpeta de imagenes o sin Pillow.")
else:
    files_all = [f for f in os.listdir(RAIZ_IMG) if f.lower().endswith(".jpg")]
    muestra = random.sample(files_all, min(3000, len(files_all)))
    dims, anchos, altos, kbs = Counter(), [], [], []
    for i, f in enumerate(muestra, 1):
        with Image.open(RAIZ_IMG / f) as im:
            dims[(im.width, im.height)] += 1
            anchos.append(im.width)
            altos.append(im.height)
            kbs.append(os.path.getsize(RAIZ_IMG / f) / 1024)
        if i % 1000 == 0:
            print(f" {i}/{len(muestra)}")
    tot = sum(dims.values())
    top = dims.most_common(6)
    print("Muestra:", tot, "imagenes")
    print("Tamaños distintos en la muestra:", len(dims))
    print("Mas frecuentes:", [(f"{w}x{h}", f"{100 * c / tot:.1f} %") for (w, h), c in top])
    print(f"Mediana ancho x alto: {np.median(anchos):.0f} x {np.median(alto):.0f}")
    print(f"Mediana de peso: {np.median(kbs):.1f} KB | min {min(kbs):.1f} | max {max(kbs):.1f}")
```

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
axes[0].hist(anchos, bins=40, color="#4575B4", edgecolor="black", linewidth=0.4)
axes[0].set_title("Ancho de las imágenes (px)")
axes[0].set_xlabel("px")
axes[1].hist(kbs, bins=40, color="#1A9850", edgecolor="black", linewidth=0.4)
axes[1].set_title("Peso de archivo (KB)")
axes[1].set_xlabel("KB")
plt.tight_layout()
plt.show()
```

1000/3000

2000/3000

3000/3000

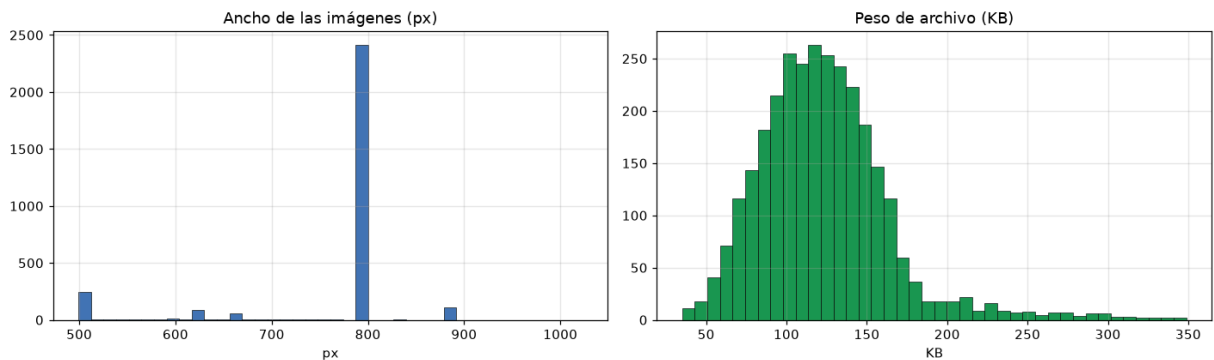
Muestra: 3000 imagenes

Tamaños distintos en la muestra: 132

Mas frecuentes: [('800x600', '80.4 %'), ('888x500', '3.7 %'), ('500x666', '3.3 %'), ('625x500', '2.8 %'), ('666x500', '1.9 %'), ('500x500', '1.7 %')]

Mediana ancho x alto: 800 x 600

Mediana de peso: 119.5 KB | min 35.1 | max 348.8 KB



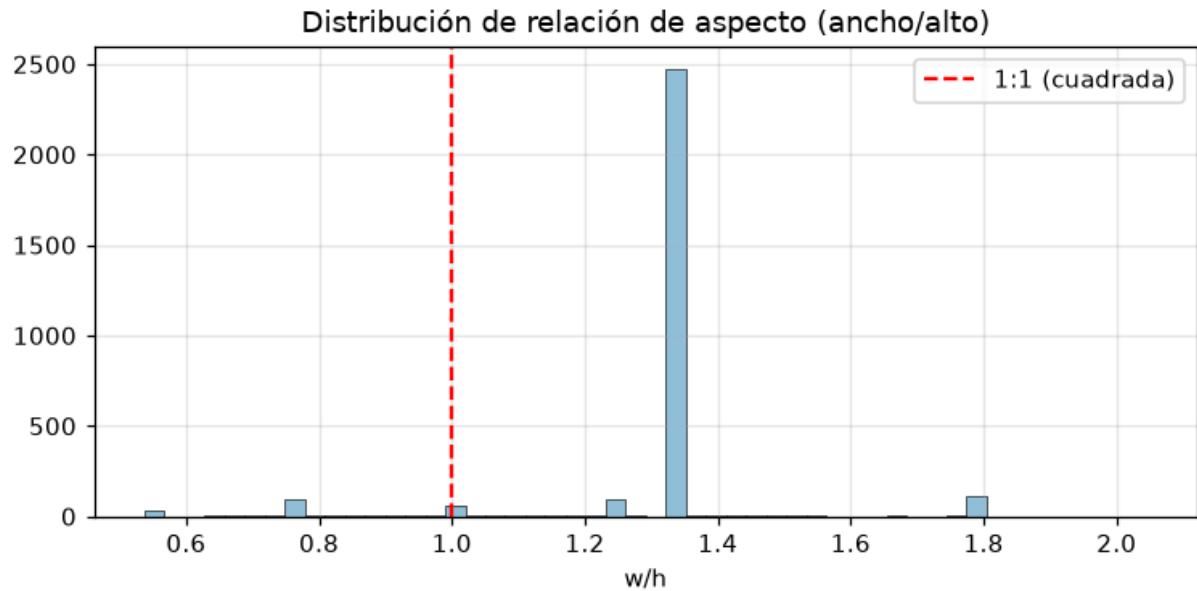
Conclusión práctica (Fase 5.1–5.2): la mayoría son 800×600 → para la red conviene **redimensionar a 224×224**. Como solo 3.4 % son casi cuadradas, un `resize` directo *deforma* la hoja; mejor `Resize(256)` con proporción + `CenterCrop(224)` (o *letterbox* con pad).

```
In [13]: if not disp_img:
          print("OMITIDO.")
        else:
          ratios = [w / h for w, h in zip(anchos, altos)]
          casi_cuadradas = np.mean([(0.9 <= r <= 1.1) for r in ratios])
          print(f"Proporción media (alto/ancho invertido->w/h): {np.mean(ratios):.3f}")
          print(f"Imágenes casi cuadradas (0.9 <= w/h <= 1.1): {100 * casi_cuadradas:.1f}")

          fig, ax = plt.subplots(figsize=(7, 3.6))
          ax.hist(ratios, bins=50, color="#91BFDB", edgecolor="black", linewidth=0.4)
          ax.axvline(1.0, color="red", linestyle="--", label="1:1 (cuadrada)")
          ax.set_title("Distribución de relación de aspecto (ancho/alto)")
          ax.set_xlabel("w/h")
          ax.legend()
          plt.tight_layout()
          plt.show()
```

Proporción media (alto/ancha invertido->w/h): 1.303

Imágenes casi cuadradas ($0.9 \leq w/h \leq 1.1$): 3.4 %



5.3 Brillo (media) y contraste (desviación estándar) por clase

n = 150 imágenes por clase, semilla fija. Sirve para detectar sesgo de captura (iluminación o cámara distintas **entre clases**, lo cual sería un sesgo grave).

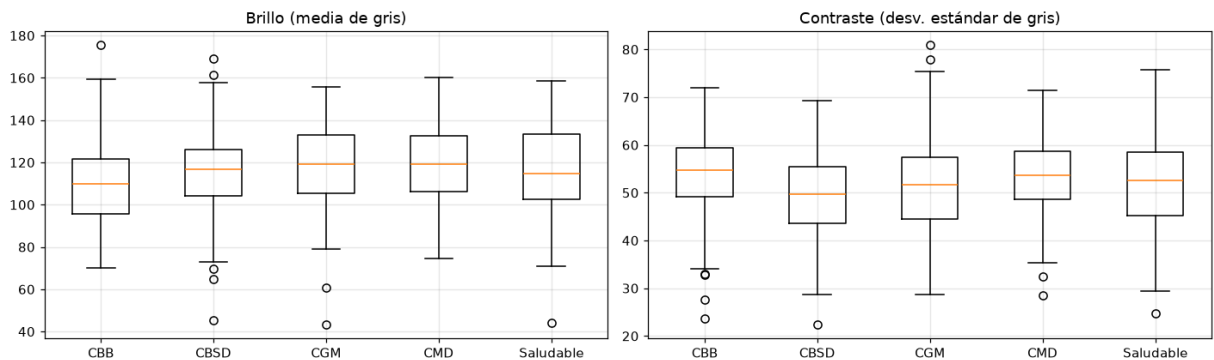
```
In [14]: if not disp_img:
          print("OMITIDO.")
        else:
            t0 = time.time()
            filas = []
            for lab in ORDEN:
                g = df.loc[df.label == lab, "image_id"]
                for f in random.sample(list(g), min(150, len(g))):
                    with Image.open(RAIZ_IMG / f) as im:
                        arr = np.asarray(im.convert("L"), dtype=np.float32)
                        filas.append({"label": lab, "archivo": f, "brillo": arr.mean(),
                                     "contraste": arr.std()})
            img_stats = pd.DataFrame(filas)
            resumen = img_stats.groupby("label")[["brillo", "contraste"]].agg(["mean", "std"])
            resumen.index = [NOMBRE[i] for i in resumen.index]
            display(resumen)
            print(f"\n total: {len(img_stats)} | tiempo {time.time() - t0:.1f} s")

            fig, axes = plt.subplots(1, 2, figsize=(13, 4))
            datos_b = [img_stats.loc[img_stats.label == lab, "brillo"] for lab in ORDEN]
            datos_c = [img_stats.loc[img_stats.label == lab, "contraste"] for lab in ORDEN]
            for ax, datos, titulo in ((axes[0], datos_b, "Brillo (media de gris)",
                                       (axes[1], datos_c, "Contraste (desv. estándar de gris)")),
                                     (axes[1], datos_c, "Contraste (desv. estándar de gris)")):
                ax.boxplot(datos)
                ax.set_xticklabels([NOMBRE[i].split(" ")[0] for i in ORDEN])
                ax.set_title(titulo)
            plt.tight_layout()
            plt.show()
```

```
print("Rango de brillo medio entre clases:",
      f"{img_stats.groupby('label').brillo.mean().min():.1f}",
      "-", f"{img_stats.groupby('label').brillo.mean().max():.1f}",
      "| Rango de contraste:", f"{img_stats.groupby('label').contraste.mean().m",
      "-", f"{img_stats.groupby('label').contraste.mean().max():.1f}")
```

	brillo		contraste	
	mean	std	mean	std
CBB (bacteriosis)	110.629997	18.680000	53.540001	8.61
CBSD (rayas marrones)	114.650002	19.080000	49.480000	8.66
CGM (moteado verde)	117.949997	18.770000	51.189999	9.86
CMD (mosaico)	117.949997	17.510000	53.560001	7.97
Saludable	116.760002	20.280001	51.700001	9.79

n total: 750 | tiempo 5.7 s



Rango de brillo medio entre clases: 110.6 - 118.0 | Rango de contraste: 49.5 - 53.6

5.4 Montajes visuales por clase (obligatorio)

Respuesta a la consigna: «*determinar si sirve toda la información de las imágenes*» — se muestran muestras aleatorias con **hoja completa, fondo/tallo y bordes**, para decidir qué regiones aportan síntoma y cuáles son ruido (al momento de elegir *augmentations* y ROI).

```
In [15]: if not disp_img:
          print("OMITIDO.")
        else:
          fig, axes = plt.subplots(len(ORDEN), 5, figsize=(17, 4.2 * len(ORDEN)))
          for r, lab in enumerate(ORDEN):
            muestras = random.sample(list(df.loc[df.label == lab, "image_id"]), 5)
            for c, f in enumerate(muestras):
              with Image.open(RAIZ_IMG / f) as im:
                axes[r, c].imshow(im)
                axes[r, c].axis("off")
              if r == 0:
                axes[r, c].set_title(f"\n{NOMBRE[lab]}", fontsize=9, color=PALETA[1])
              if c == 0:
                axes[r, c].text(-0.15, 0.5, NOMBRE[lab], rotation=90,
                                transform=axes[r, c].transAxes, va="center",
```

```

        fontsize=10, color=PALETA[lab], fontweight="bold")
    for c in range(len(muestras), 5):
        axes[r, c].axis("off")
    fig.suptitle("Muestras aleatorias por clase (semilla 42)", y=1.005, fontsize=13)
    plt.tight_layout()
    plt.show()

```

Muestras aleatorias por clase (semilla 42)



5.5 Imágenes no informativas (reportar, no borrar sin decisión documentada)

Percentiles de brillo y contraste sobre una muestra independiente (n = 1,200, semilla 42): se listan las candidatas a casi-negras, casi-blancas o planas (poco detalle).

```
In [16]: if not disp_img:
        print("OMITIDO.")
    else:
        t0 = time.time()
        files_all = [f for f in os.listdir(RAIZ_IMG) if f.lower().endswith(".jpg")]
        muestra = random.sample(files_all, min(1200, len(files_all)))
        filas = []
        for f in muestra:
            with Image.open(RAIZ_IMG / f) as im:
                arr = np.asarray(im.convert("L"), dtype=np.float32)
                filas.append({"archivo": f, "brillo": arr.mean(), "contraste": arr.std()})
        s = pd.DataFrame(filas)
        p01, p99 = np.percentile(s.brillo, [1, 99])
        casi_negras = s[s.brillo < p01].sort_values("brillo")
        casi_blancas = s[s.brillo > p99].sort_values("brillo", ascending=False)
        planas = s[s.contraste < np.percentile(s.contraste, 1)].sort_values("contraste")
        print(f"n = {len(s)} | percentiles brillo p01={p01:.1f} p99={p99:.1f}")
        print(f"Candidatas casi-negras (<p01): {len(casi_negras)} | casi-blancas (>p99)
              f" | planas (contraste < p01): {len(planas)}")
        print("\nEjemplos mas planos:")
        display(planas.head(5).round(1))
        print(f"Tiempo {time.time() - t0:.1f} s")
        print("Decision: NO se elimina ninguna imagen en el EDA; se deja listado para l
```

```
n = 1200 | percentiles brillo p01=72.6 p99=157.3
Candidatas casi-negras (<p01): 12 | casi-blancas (>p99): 12 | planas (contraste < p0
1): 12
```

Ejemplos mas planos:

	archivo	brillo	contraste
679	277532565.jpg	91.699997	18.100000
419	train-cbsd-646.jpg	133.300003	21.799999
729	1485753217.jpg	137.000000	22.700001
715	2892378347.jpg	142.899994	23.799999
89	4179630738.jpg	158.399994	24.700001

Tiempo 8.7 s

Decision: NO se elimina ninguna imagen en el EDA; se deja listado para la etapa de p
reprocesamiento.

Fase 6 — Propuesta de split y preprocesamiento

Split **estratificado por clase** (70/15/15, semilla 42) y verificación de que `source` también quede repartido en proporción dentro de cada split (evita el sesgo de dominio 2019 vs 2020).

```
In [17]: def split_estratificado(df, semilla=42, prop=(0.70, 0.15, 0.15)):
    out = []
    for lab, g in df.groupby("label"):
        idx = g.index.to_numpy().copy()
        rng = np.random.default_rng(semilla)
        rng.shuffle(idx)
        n = len(idx)
        n1 = int(round(prop[0] * n))
        n2 = int(round(prop[1] * n))
        out.append(pd.DataFrame({
            "split": ["entrenamiento"] * n1 + ["validacion"] * n2 + ["test"] * (n -
            "image_id": idx,
        })))
    res = pd.concat(out, ignore_index=True).set_index("image_id")
    return res

split = split_estratificado(df)
dfx = df.join(split)

print("n por split:")
display(dfx.split.value_counts().to_frame("n").join((100 * dfx.split.value_counts(n

ct_s = pd.crosstab(dfx.label, dfx.split, margins=True, margins_name="Total")
ct_s.rename(index=NOMBRE, inplace=True)
print("Estratificacion por CLASE (n):")
display(ct_s)

pct_s = (pd.crosstab(dfx.label, dfx.split, normalize="columns") * 100).round(2)
pct_s.rename(index=NOMBRE, inplace=True)
print("% dentro de cada split (debe parecerse a la distribucion global):")
display(pct_s)

ct_src = pd.crosstab(dfx.source, dfx.split, margins=True, margins_name="Total")
print("Reparto de source por split (n) - debe conservar ~81/19 %:")
display(ct_src)
print("Participacion de 2019 por split (%):")
display((100 * pd.crosstab(dfx.source, dfx.split, normalize="columns")).loc[[2019])
```

n por split:

	n	%
split		
entrenamiento	18435	70.0
test	3951	15.0
validacion	3951	15.0

Estratificacion por CLASE (n):

	split	entrenamiento	test	validacion	Total
label					
CBB (bacteriosis)		1044	224	224	1492
CBSD (rayas marrones)		2433	522	521	3476
CGM (moteado verde)		2112	452	453	3017
CMD (mosaico)		10823	2320	2319	15462
Saludable		2023	433	434	2890
Total		18435	3951	3951	26337

% dentro de cada split (debe parecerse a la distribucion global):

	split	entrenamiento	test	validacion
label				
CBB (bacteriosis)		5.66	5.67	5.67
CBSD (rayas marrones)		13.20	13.21	13.19
CGM (moteado verde)		11.46	11.44	11.47
CMD (mosaico)		58.71	58.72	58.69
Saludable		10.97	10.96	10.98

Reparto de source por split (n) – debe conservar ~81/19 %:

	split	entrenamiento	test	validacion	Total
source					
2019		3408	768	764	4940
2020		15027	3183	3187	21397
Total		18435	3951	3951	26337

Participacion de 2019 por split (%):

	split	entrenamiento	test	validacion
source				
2019		18.49	19.44	19.34

Reglas anti-fuga y preprocesamiento propuesto

1. **Sin duplicados entre splits:** la Fase 2.1 verificó 0 duplicados exactos; si en el futuro aparecen *near-duplicates*, se agrupan por hash perceptual **antes** de partir.
2. `image_id` (y por ende el nombre `train-<clase>-...`) **nunca** entra como feature: solo píxeles.

- Entrada de red: `Resize(256) + CenterCrop(224)` (no `resize(224,224)` directo: deforma).
- Normalización ImageNet (backbones con transfer learning): `mean=[0.485, 0.456, 0.406]` , `std=[0.229, 0.224, 0.225]` .
- Augmentations* candidatas (coherentes con lo visto en 5.3): flip horizontal (la hoja no tiene orientación semántica), rotación $\pm 15^\circ$, brillo/contraste $\pm 10\%$, desplazamiento; **no** usar cambios de color fuertes si los síntomas dependen del color (CMD/CGM justamente).
- Clases desbalanceadas: `class_weight="balanced"` o *oversampling* dentro del split de **entrenamiento** jamás antes.

Fase 7 — Conclusiones

Verificación: todos los números de esta sección se contrastaron con la salida ejecutada de las celdas anteriores (regla anti-divergencia de la instrucción).

Resumen

Dimensión	Resultado
Registros	26,337 × 3 (<code>image_id</code> , <code>label</code> , <code>source</code>); sin nulos; sin <code>image_id</code> duplicados
Imágenes	26,337 <code>.jpg</code> en carpeta plana, 1:1 con el CSV (0 faltantes / 0 sobrantes); ≈ 3.09 GiB
Duplicados exactos	0 (SHA-1 completo de los 26,337 archivos)
Legibilidad	0 imágenes corruptas en la muestra de 3,000; todas en modo RGB
Clases (n y % sobre el total)	CBB 1,492 = 5.67 % · CBSD 3,476 = 13.20 % · CGM 3,017 = 11.46 % · CMD 15,462 = 58.71 % · Saludable 2,890 = 10.97 %
Desbalance	ratio máx/mín = 10.36 : 1 (CMD vs CBB)
Fuentes	2020: 21,397 = 81.24 % · 2019: 4,940 = 18.76 %
Filtrado por nombre	las 4,940 de 2019 son <code>train-<clase>-NNN.jpg</code> con 100 % de coherencia con <code>label</code> → no usar el nombre como feature
Dimensiones	dominante 800×600 80.4 % del muestreo), 132 tamaños distintos, mediana 119.5 KB
Aspecto	media w/h = 1.30; solo 3.4 % casi cuadradas → <code>Resize(256)+CenterCrop(224)</code>
Brillo/contraste por clase	promedios casi idénticos entre clases (brillo 110.6–118.0, contraste 49.5–53.6) → sin sesgo de captura por clase
Imágenes no informativas	listadas por percentiles; no se borra ninguna sin decisión documentada

Decisiones

1. **Métricas del trabajo:** macro-F1, balanced accuracy, ROC-AUC/PR-AUC y matriz de confusión normalizada por fila; el accuracy crudo se reporta solo como referencia.
2. **Costo asimétrico:** penalizar más el error Saludable ↔ Enferma (consigna de la cátedra).
3. **Split estratificado por clase** con semilla 42 (70/15/15), verificado también por `source` para mitigar el sesgo de dominio entre 2019 y 2020.
4. **Preprocesamiento:** `Resize(256) + CenterCrop(224)`, normalización ImageNet, aumentaciones acotadas (sin alterar fuertemente el color).

Pendientes / lo que falta

- Decisión de política sobre imágenes muy planas o casi monocromáticas (listadas en 5.5).
- Evaluar *near-duplicates* con hash perceptual (pHash) antes del split definitivo.
- Confirmar con muestra visual si el fondo/tallo/raíz aportan síntoma o solo ruido (montajes 5.4).

Fase 8 — Reproducibilidad y entregables

Regla de carpetas: el código (`.ipynb` , `.py`) vive en la **raíz**; todo PDF/HTML de resumen o explicación va a `reportes/`; los `.html` **solo se generan si se piden explícitamente**.

Google Colab: subir `labels.csv` (celda 2 lo detecta); para las celdas de imágenes subir `mandioca.zip` y ejecutar la celda siguiente.

Ejecución desde terminal (gate de calidad):

```
python -m nbconvert --to notebook --execute --inplace --
ExecutePreprocessor.timeout=1800 01_EDA_mandioca.ipynb
python -m nbconvert --to script 01_EDA_mandioca.ipynb
python -m nbconvert --to html --embed-images --output-dir reportes
01_EDA_mandioca.ipynb # solo si se pide
# PDF desde el HTML con Edge headless (no hay pandoc):
# msedge --headless --no-pdf-header-footer --print-to-
pdf=".../reportes/01_EDA_mandioca.pdf"
"file:///.../reportes/01_EDA_mandioca.html"
```

```
In [18]: # Colab: descomentar para subir el dataset de imagenes y descomprimirlo.
# import zipfile
# from google.colab import files
# upload = files.upload() # subir mandioca.zip
# with zipfile.ZipFile(list(upload)[0]) as z:
#     z.extractall("datos")
# print("Listo")
```

```
In [19]: import nbformat
_nbk = nbformat.read(str(Path(r"C:\Users\Fabricio\Desktop\final_inteligencia\01_EDA
print("Celdas:", len(_nbk.cells),
```

```
"| markdown:", sum(c.cell_type == "markdown" for c in _nbk.cells),  
"| código:", sum(c.cell_type == "code" for c in _nbk.cells))
```

Celdas: 38 | markdown: 19 | código: 19